Digital Communications

ECE 2564, Embedded Systems

Fall 2025

Motivation and Terminology

- How to communicate with other devices and boards?
  - Simplex: data always goes in one direction only – from a transmitter to a receiver
  - Half Duplex: data can travel in only one direction at a time, like a walkie-talkie
  - Full Duplex: data can travel in both directions simultaneously

  - Serial: send/receive a single bit at a time
  - Parallel: send/receive multiple bits in the same direction simultaneously

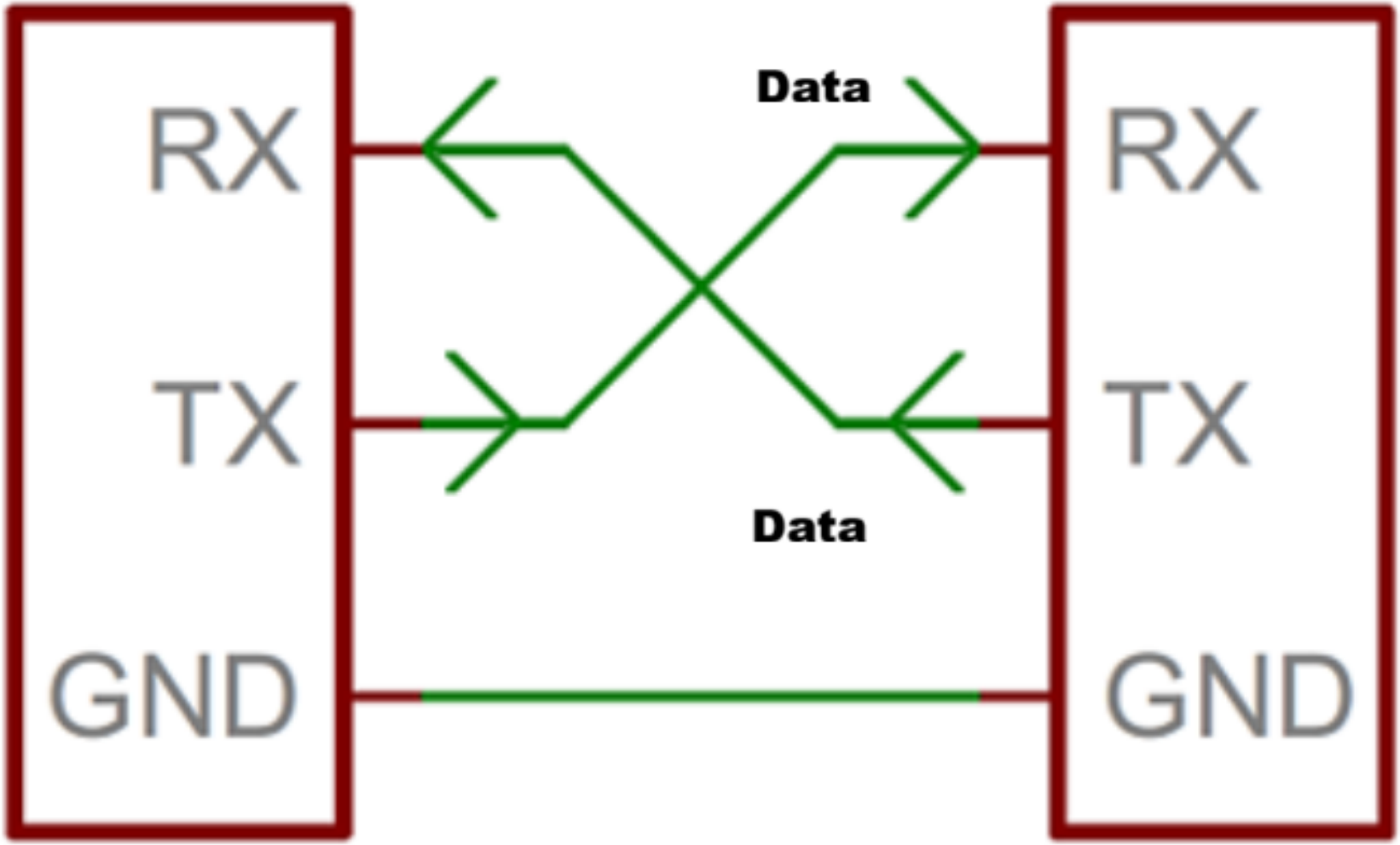
  - Synchronous: the parties share a clock signal
  - Asynchronous: no common clock signal
- Listing of common communication protocols
  - UART = universal asynchronous receiver-transmitter
  - SPI = serial peripheral interface,
  - I^2C = Inter-integrated circuit

Serial Communication

- Synchronous vs. Asynchronous
  - For *synchronous transmission* the sender and receiver share a clock signal
    - For example, the *sender* might always provide a timing signal that the *receiver* uses
  - In an *asynchronous transmission* the sender and receiver do not share a clock signal
    - This is often used when devices are not in close proximity. Why?
  - NOTE: Not every communication system that contains clocks is synchronous

Commonly Used Standards for Communication

UART

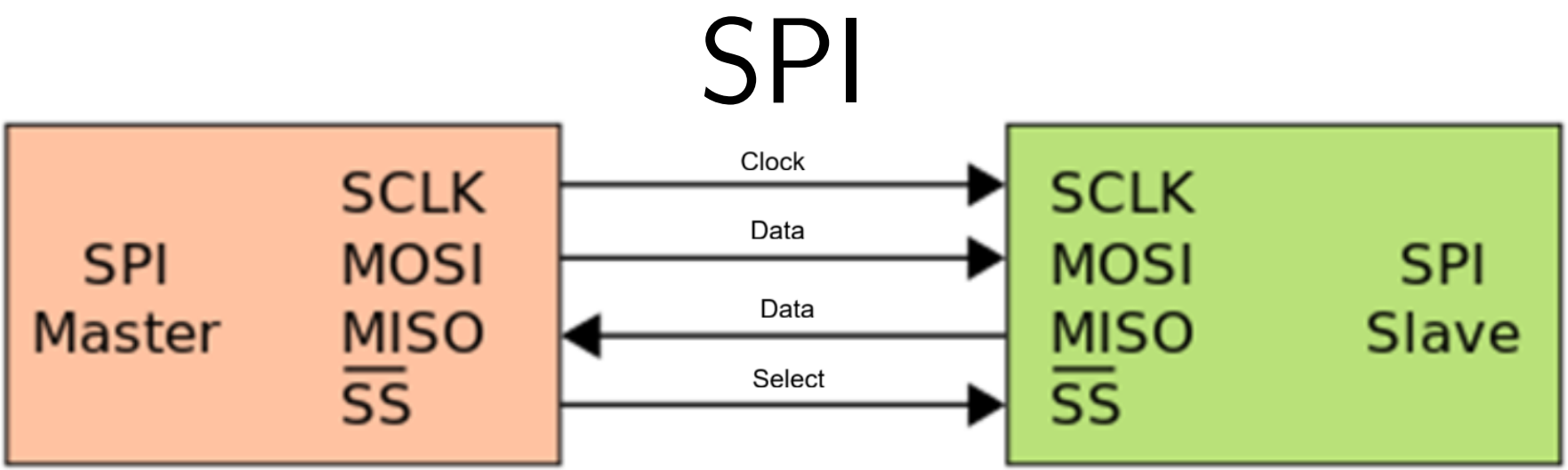


In the Old Days ...

Serial communication handled by RS-232

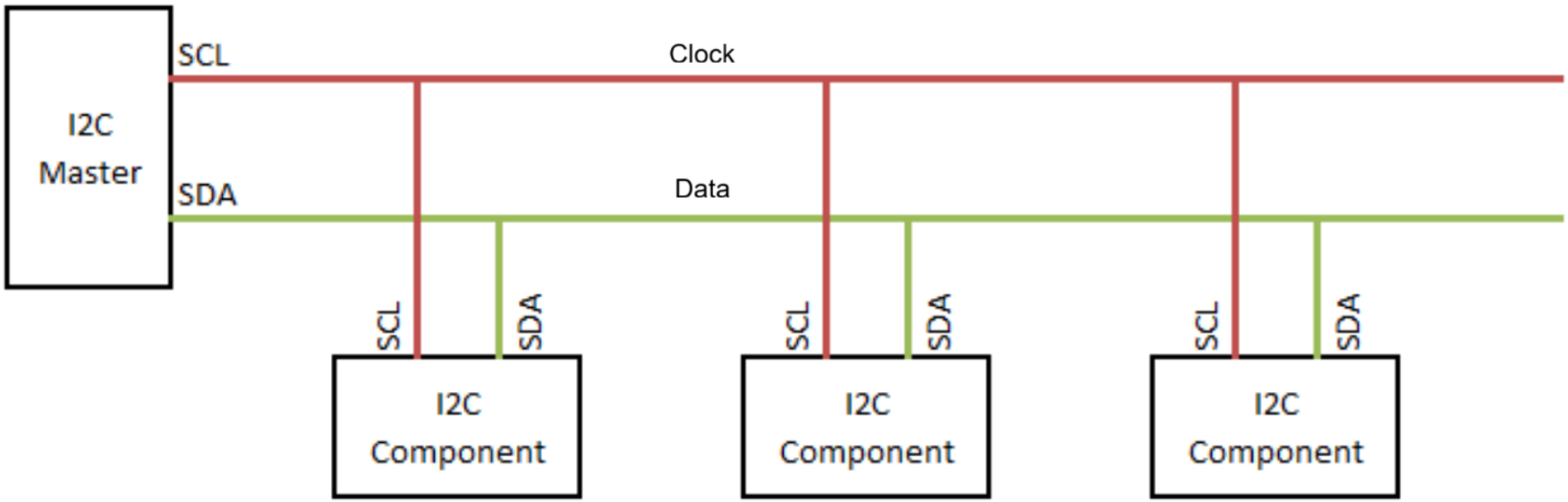


Commonly Used Standards for Communication



Commonly Used Standards for Communication

I^2C



Communication Speed

- Bit rate: The number of bits transferred per second, including control bits and other non-data bits
- Baud rate: The number of *symbols* transferred per second
  - When the symbols are 0 and 1 the baud rate equals the bit rate
  - In some systems a symbol might be n bit, for example a `char` is 8 bits, and then bit rate is equal to n x baud rate
- Data rate: Data bits transferred per second, excluding non-data bits
  - The data rate is usually lower than the bit rate

Error Detection

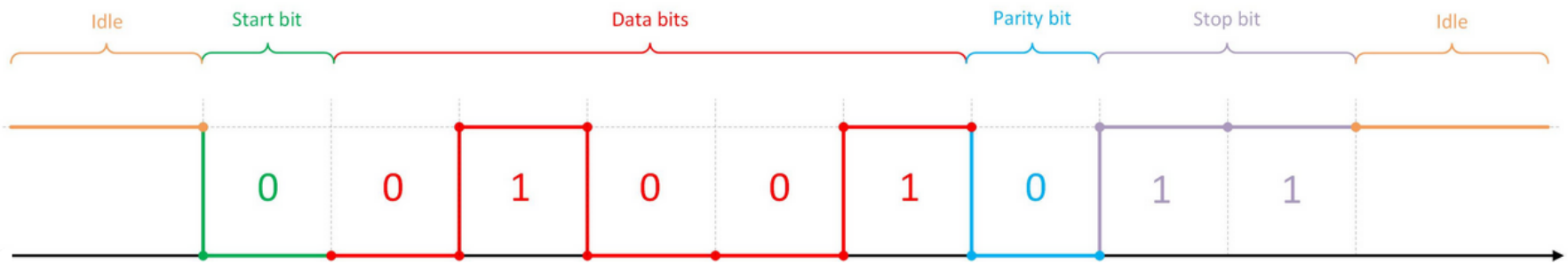
- Parity
  - Odd: total number of 1s should be odd, including the parity bit
  - Even: total number of 1s should be even, including the parity bit
  - Example data: 10110100
    - For odd parity: append 1 for this example
    - For even parity: append 0 for this example
- Other types of error detection and error correction exist but are out of scope for this course
  - CRC, Hamming codes, etc.

Serial Communication

- A microcontroller typically operates on data values that are of size 8 bit, 16 bits or 32 bits each
- To perform serial communication the device needs to:
  - Perform a *parallel-to-serial* conversion
  - Transmit the data serially
  - Perform a *serial-to-parallel* conversion at the receiving end

Universal Asynchronous Receiver-Transmitter: UART

- Serial + asynchronous + full duplex
- Typical bit stream (UART *frame*)
  - Idle bits: logic 1
  - Start bit: logic 0
  - Data bits: logic 0 or 1 (configurable for 7 or 8 bits)
  - Parity bit (optional): odd or even (configurable)
  - Stop bits: 1 or 2 bits (configurable)
  - The receiver detects the *start* of a frame by observing the 1-to-0 transition
  - Frame size: between 9 and 12 bits, inclusive



Representative Timing Diagram (5 data bits due to space limitation)

UART Configuration

- The number of start bits is constant and not configurable
- Number of data bits
- Existence and type of parity
- Number of stop bits
- LSB or MSB

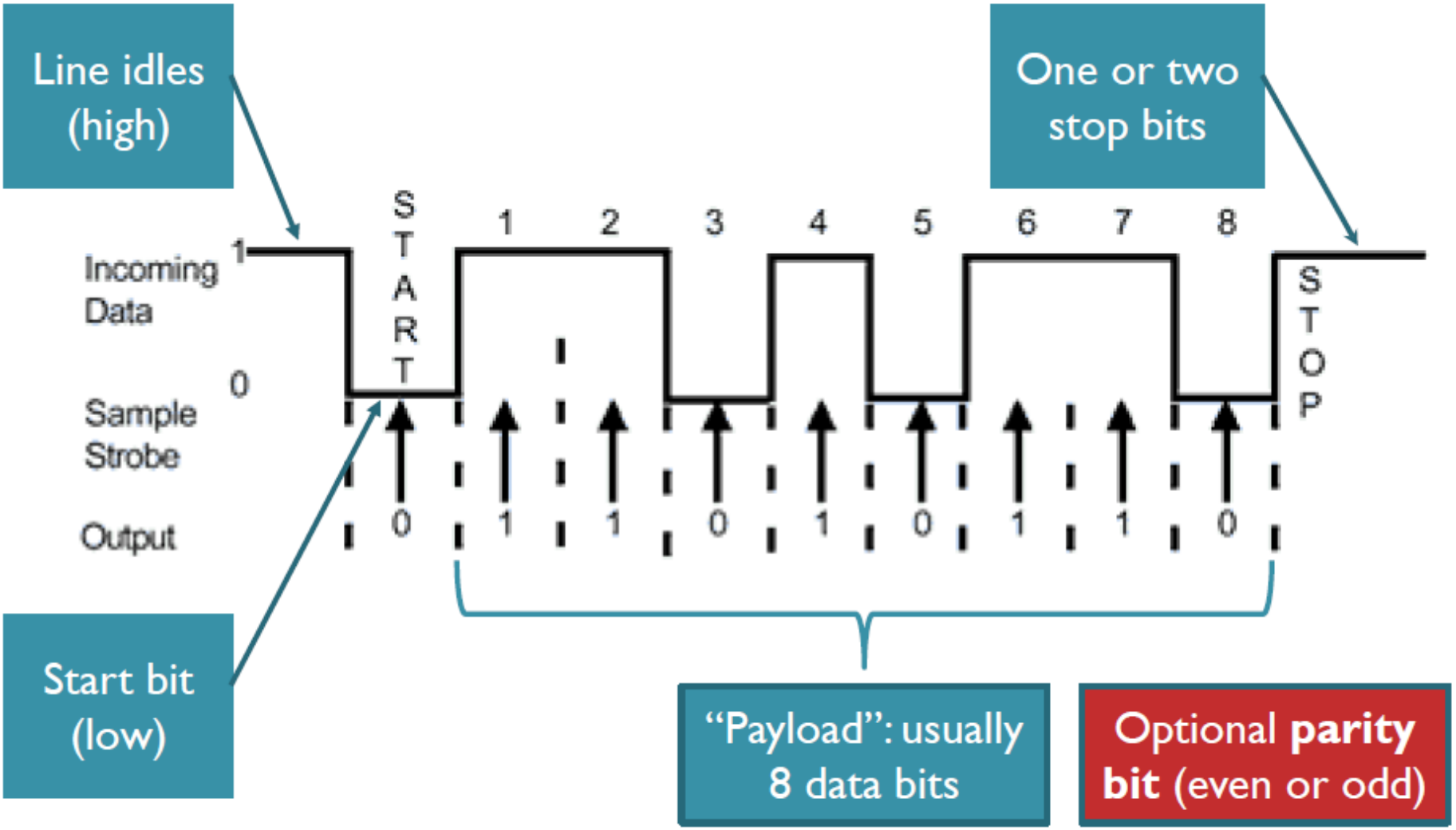
UART Configuration

Bit title	Number of bits	Value
start	1	logic 0
data	5-9[1]	depends on data
parity	0-1	depends on data and even/odd configuration
stop	1-2	logic 1

Example: You may see the configuration 7E1: 7 data bits, Even parity, 1 stop bit

[1] The number of data bits in MSP432 UART is 7 or 8. In general embedded systems can have 5 to 9 data bits.

Data Frames


Line idles (high)
One or two stop bits
Incoming Data
Sample Strobe
Output
1
0
START
STOP
1 2 3 4 5 6 7 8
0 1 1 0 1 0 1 1 0
Start bit (low)
"Payload": usually 8 data bits
Optional **parity bit** (even or odd)

Agreement on Data Frames

- The devices involved in the communication must agree to certain things **in advanced**
  - The *rate* at which bits will be transmitted
  - The number of data bits per frame
  - Whether or not *parity* will be used and if so, what kind of parity the communication will use
  - The number of *stop bits*
- When we configure the UART module we must provide all these parameters
- When we configure a terminal program, like our laptops, we must make sure that the parameters agree with those of the MSP432

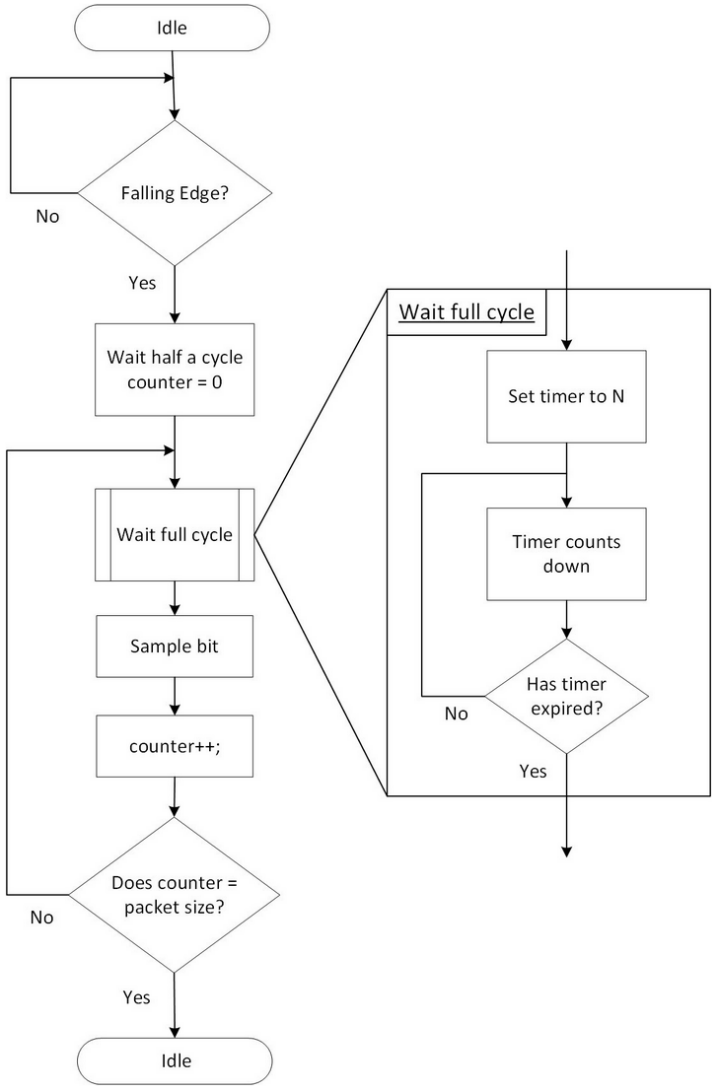
How to Receive Data

- UART is asynchronous. How do we "get the data" from the bitstream?
  - Need to know the baud rate or bit rate
  - Need to detect the beginning of a frame based on the falling edge of the start bit
  - We then must *sample* the received signal to obtain the individual bits
- If the data rate is 9600 Bits Per Second (BPS) how do we obtain the bits?
  - The duration of each bit is $1/9600\,\mathrm{s} \approx 104\,\mu\mathrm{s}$
  - If transmitting with no idle time, 9600 bits are transferred every second
- When is the best time to sample a data bit?
  - the beginning of the bit?
  - the middle of the bit?
  - the end of the bit?

How to Receive Data

- If the bit rate is 10,000 BPS, what is the duration of a bit?
  - $1/10\,000\,\text{s} = 0.1\,\text{ms}$
- How does the UART wait for the length of a bit?
  - It uses a count-down counter to measure time. We assume:
    - The counter counts down by 1 every T microseconds
    - The counter is initially loaded with N, the *division factor*
- How long does it take the counter to go from N to 0?
  - N*T microseconds
- The count-down counter uses a clock signal called *baud rate counter* (BRCLK) with a frequence of f Hz to count down
  - How long does it take for the counter to expire in terms of f?
    - Answer: N/f
    - Example: $f = 100\,\text{MHz}$ and if N=100, then it will take $1\,\mu\text{s}$

Flow Chart for Sampling



Another Example

- $f = 3$ MHz, B = 9600 BPS
- So N $= f/$B $= 3000000/9600 = 312.5$
- Should we choose N=312 or N=313?

When the wait time is shorter than the ideal sampling time:

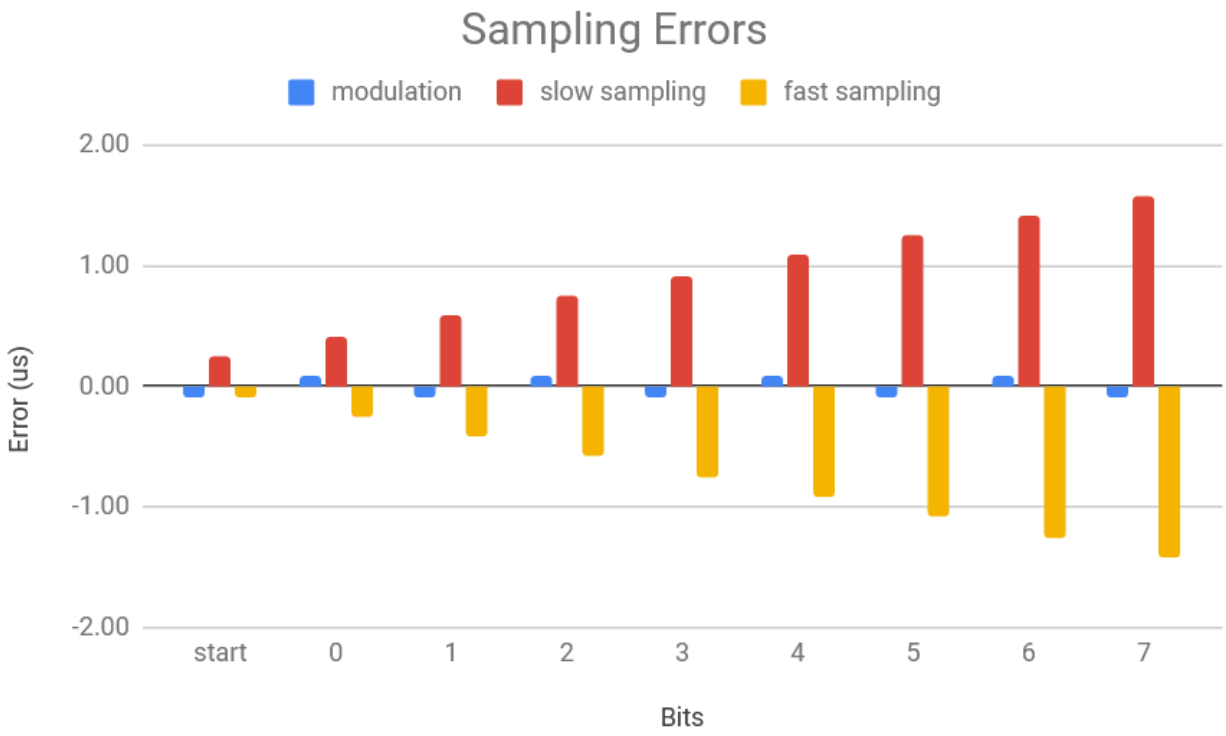


When the wait time is longer than the ideal sampling time:



Error Drift in Sampling Time

Spreadsheet Model for Sampling Time Error Drift



Potential Modulation Trick

- If N=312.5
  - 312, 313, 312, 313, 312, 313, 312, 313
- If N=312.25
  - 312, 312, 312, 313, 312, 312, 312, 313
- What about N=312.108?

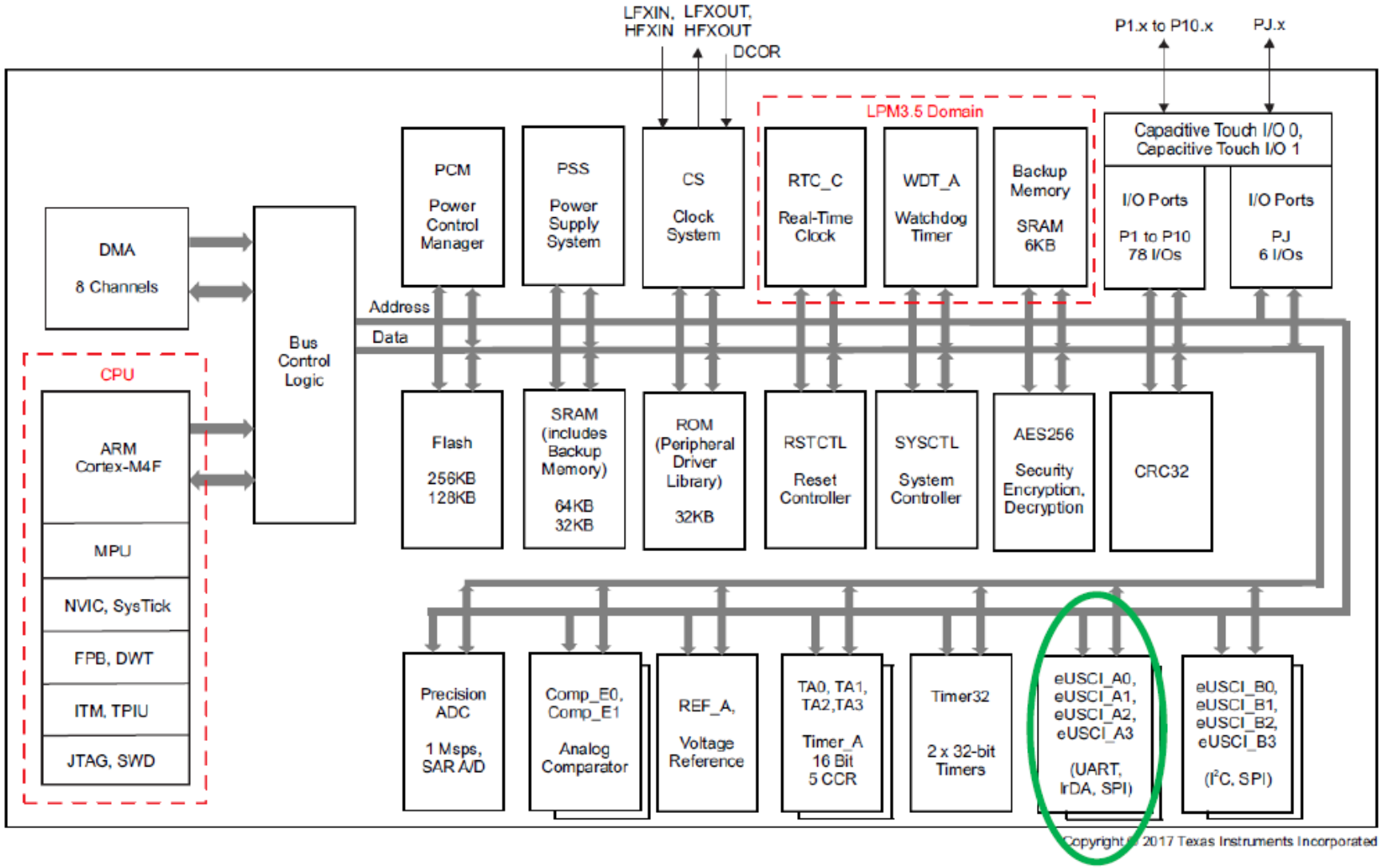
Coming Attractions: Handling the Fractional Part

Table 24-4. UCBRSx Settings for Fractional Portion of N = f_{BRCLK}/Baud Rate

Fractional Portion of N	UCBRSx[1]		Fractional Portion of N	UCBRSx[1]
0.0000	0x00		0.5002	0xAA
0.0529	0x01		0.5715	0x6B
0.0715	0x02		0.6003	0xAD
0.0835	0x04		0.6254	0xB5
0.1001	0x08		0.6432	0xB6
0.1252	0x10		0.6667	0xD6
0.1430	0x20		0.7001	0xB7
0.1670	0x11		0.7147	0xBB
0.2147	0x21		0.7503	0xDD
0.2224	0x22		0.7861	0xED
0.2503	0x44		0.8004	0xEE
0.3000	0x25		0.8333	0xBF
0.3335	0x49		0.8464	0xDF
0.3575	0x4A		0.8572	0xEF
0.3753	0x52		0.8751	0xF7
0.4003	0x92		0.9004	0xFB
0.4286	0x53		0.9170	0xFD
0.4378	0x55		0.9288	0xFE

(1) The UCBRSx setting in one row is valid from the fractional portion given in that row until the one in the next row

UARTs and the MSP432



Serial Communication with MSP432

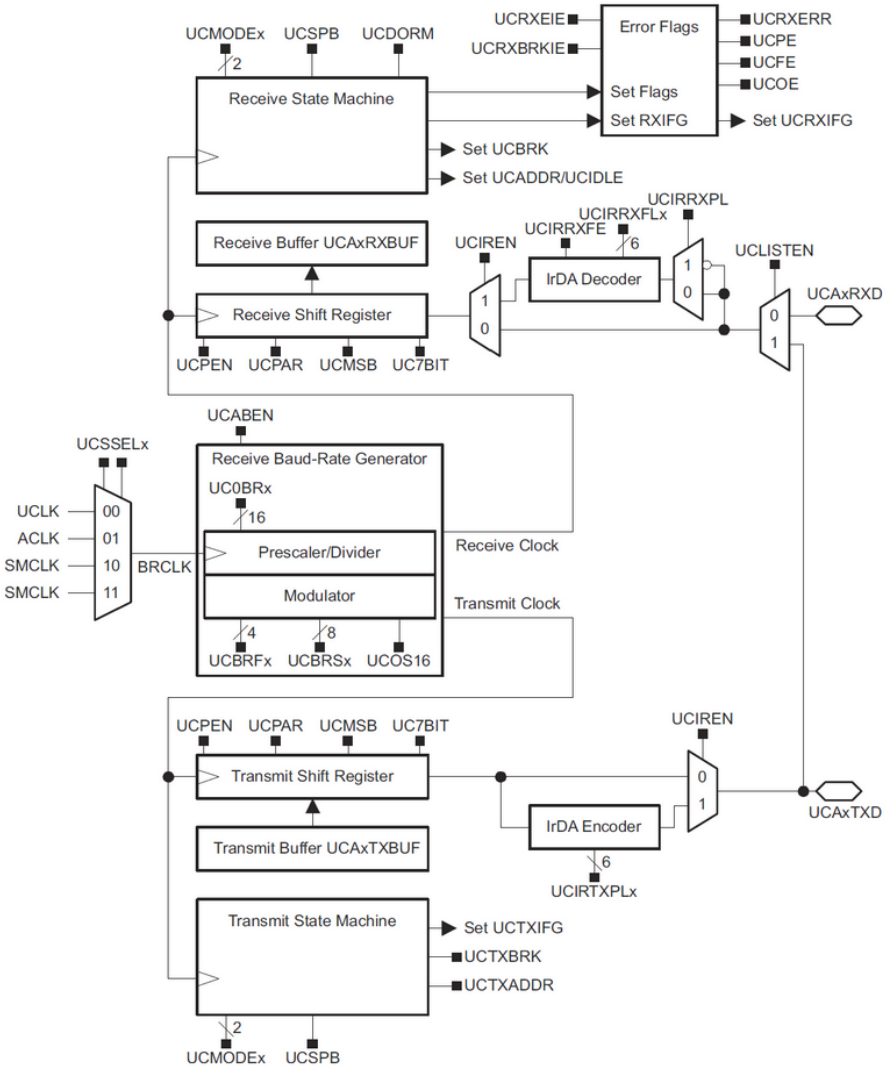
- The MSP432 provides several subsystems that TI calls *Enhanced Universal Serial Communication Interface* (eUSCI)
- The eUSCI peripherals have been designed to support several serial communication standards

UART	$\longleftarrow$	Learning now
SPI	$\longleftarrow$	More about this later this semester
I^2C	$\longleftarrow$	More about this later this semester
IrDA	$\longleftarrow$	Not covered in this class

Serial Communication with MSP432

- The MSP432P401x MCUs contain up to 4 eUSCI_A modules and 4 eUSCI_B modules
- eUSCI_An: provides support for UART, enhanced UART, SPI and IrDA
- eUSCI_Bn: provides support for SPI and I^2C

eUSCI_Ax Block Diagram – UART Mode (UCSYNC=0)



In each direction, transmit and receive, the heart of the UART is a shift register

Programming the UART – Coming Attractions

```
\\ Initialize a UART Subsystem

bool UART_initModule( uint32_t moduleInstance,
                      const eUSCI_UART_Config *config);


void UART_enableModule (uint32_t moduleInstance);
```

Programming the UART – Coming Attractions

```
// The following can be used to check whether the receiver has received
// a character, and whether the transmitter is ready to transmit another
// character
uint_fast8_t UART_getInterruptStatus ( uint32_t moduleInstance,
                                       uint8_t mask );

// Transfer a received character from the UART to the CPU
uint8_t UART_receiveData ( uint32_t moduleInstance );

// Transfer a character from the CPU to the UART;
// by default, the UART will begin immediately to transmit the character
void UART_transmitData ( uint32_t moduleInstance,
                         uint_fast8_t transmitData )
```

Summary

- Introduction to basics of digital communication
- Serial versus Parallel
- What is serial communication popular?
- Simplex; Half Duplex; Full Duplex — Which allows simultaneous transmission?
- Synchronous versus Asychronous
- Error detection using parity bit
- Communication speed
- Sampling data
- Communication Standards: UART, SPI, I^2C

Summary

- UART: Universal Asynchronous Receiver-Transmitter
- A UART is a peripheral device that interfaces between a serial communication link and the (parallel) data bus of the CPU